FOXLINK GPT ↔ MCP 連接規格書

> ***給 MCP 伺服器開發者*** *— 本文件說明如何接受 FOXLINK GPT 的呼叫,並透過* `X-User-Token` *取得可信的使用者身份以做資料權限判斷。*
>
> ***版本****:v1.0(2026-04-17)* ***狀態****:FOXLINK GPT 端已實作完成,可立即對接。* ***聯絡****:* `rich_lai@foxlink.com` *(實作支援、索取公鑰、測試 token)*

---

1. 概述

1.1 你要做什麼

實作一個 MCP(Model Context Protocol)伺服器,可被 FOXLINK GPT 呼叫,並:

1. 實作標準 MCP JSON-RPC 方法: `initialize` / `tools/list` / `tools/call`
2. 支援以下至少一種傳輸協定: `http-post` 、 `streamable-http` 、 `http-sse` 、 `stdio`
3. **驗證每次請求 header 中的 `X-User-Token` (RS256 JWT)** ← 本文件重點
4. 根據 JWT 中的 `email` 做資料權限判斷(例如查 AD group → 決定回傳資料範圍)

1.2 為什麼要驗證使用者身份

FOXLINK GPT 是 AI 對話平台的前端;不同使用者會透過 FOXLINK GPT 呼叫同一個 MCP 工具。你(MCP 開發者)有兩個選擇:

- **相信 FOXLINK GPT**:所有使用者都能看到所有資料(不建議)
- **自己判權(推薦)**:透過 `X-User-Token` 拿到使用者 email,自己查 AD / 自己判斷資料存取權限

此規格採**非對稱金鑰(RS256)**:FOXLINK GPT 持有**私鑰**簽發 token,你只需要一把**公鑰**來驗證。即使你的 MCP server 被攻破,攻擊者也無法偽造任何使用者身份(因為沒有私鑰)。

1.3 認證層級

一次 MCP 請求會帶兩個認證 header:

Header	代表	用途
`Authorization: Bearer <api_key>`	**服務身份**	確認「是 FOXLINK GPT 在打我」。api_key 由你自行產生,提供給 FOXLINK GPT admin 設定。
`X-User-Token: <JWT>`	**使用者身份**	確認「是哪個使用者在操作」。由 FOXLINK GPT 用 RS256 簽發,你用我們的公鑰驗簽。

> ⚠️ ***重要****:* `X-User-Token` *是****否存在****取決於 FOXLINK GPT admin 是否為這個 MCP server 開啟「傳送使用者身份」開關。對接初期建議先同意收到 token 才強制驗證,漸進啟用。*

---

2. HTTP 請求格式

2.1 範例

```
POST /mcp HTTP/1.1
Host: mcp.yourdomain.internal
Content-Type: application/json
Authorization: Bearer your-service-api-key
X-User-Token: eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOi...

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "query_sales_report",
    "arguments": { "month": "2026-03" }
  }
}
```

2.2 哪些請求會帶 `X-User-Token`

JSON-RPC Method	是否帶 X-User-Token	說明
`initialize`	可能帶,可能不帶	握手階段,建議忽略 token(即使帶了也不驗)
`tools/list`	**不帶**	服務層同步,沒有特定使用者情境
`tools/call`	**帶**(若 admin 啟用)	這是判權的主戰場

---

3. JWT 規格

3.1 Header

```
{ "alg": "RS256", "typ": "JWT" }
```

3.2 Claims

Claim	類型	範例值	說明
`iss`	string	`"foxlink-gpt"`	Issuer,固定值,必須驗證
`iat`	int	`1744689600`	簽發時 UTC epoch
`exp`	int	`1744689900`	過期時間(`iat + 300` → 5 分鐘)
`jti`	string (UUID)	`"e3b0c442-98fc-..."`	每次簽發唯一,可用於 replay 防禦 + log 對照
`sub`	string	`"12345"`	員工編號(若無則 FOXLINK GPT user id)

email	string	"peter.wang@foxlink.com"	**你判權的主要依據**
name	string	"王小明"	顯示名稱,可能為 null
dept	string	"IT-01"	部門代碼(dept_code),可能為 null

3.3 關於 `name` / `dept`

這兩個欄位是 FOXLINK GPT 從自己 DB 讀出來簽進 token 的,**可能與 AD 不同步**(例如使用者剛換部門,AD 已更新但 FOXLINK GPT DB 還沒)。

若你需要**最權威**的資訊(current AD group / dept / roles),請拿 `email` 自行查 AD,不要依賴 token 裡的 `name` / `dept`。

---

4. 驗證步驟(必做)

MCP 端收到請求後,針對 `X-User-Token` 必須完成:

1. ✅ 取出 `X-User-Token` header
2. ✅ 用 FOXLINK GPT 提供的 RSA 公鑰驗簽
3. ✅ **限定 `algorithms: ['RS256']`** — 防止 alg 切換攻擊(CVE-2016-5431)
4. ✅ 驗 `iss === "foxlink-gpt"`
5. ✅ 驗 `exp` 未過期,**容忍 30 秒 clock skew**(避免 pod 間 NTP 漂移造成假陰性)
6. ✅ 取出 `email` claim,做你的資料權限判斷

缺任何一步都可能留下安全漏洞。

---

5. 各語言驗簽範例

5.1 Node.js(`jsonwebtoken`)

```
npm install jsonwebtoken
```

```
const jwt = require('jsonwebtoken');
const fs  = require('fs');

// 啟動時載入公鑰一次
const PUBLIC_KEY = fs.readFileSync(
  process.env.FOXLINK_GPT_PUBLIC_KEY_PATH || './foxlink-gpt-public.pem'
);

function verifyUserToken(req) {
  const token = req.headers['x-user-token'];
  if (!token) throw new Error('Missing X-User-Token');

  return jwt.verify(token, PUBLIC_KEY, {
```

```
    algorithms: ['RS256'],   // 重要:鎖死 RS256,防 alg 切換
    issuer: 'foxlink-gpt',
    clockTolerance: 30,      // 30 秒 clock skew
  });
  // → { jti, sub, email, name, dept, iat, exp, iss }
}

// Express middleware 範例
function requireUser(req, res, next) {
  try {
    req.userClaims = verifyUserToken(req);
    next();
  } catch (e) {
    res.status(401).json({ error: e.message });
  }
}
```

5.2 Python(`PyJWT`)

```
pip install PyJWT cryptography
```

```
import jwt
from pathlib import Path
import os

PUBLIC_KEY = Path(
    os.getenv("FOXLINK_GPT_PUBLIC_KEY_PATH", "./foxlink-gpt-public.pem")
).read_text()

def verify_user_token(request):
    token = request.headers.get("X-User-Token")
    if not token:
        raise ValueError("Missing X-User-Token")

    return jwt.decode(
        token,
        PUBLIC_KEY,
        algorithms=["RS256"],   # 鎖死 RS256
        issuer="foxlink-gpt",
        leeway=30,              # 30 秒 clock skew
    )
    # → {"jti": "...", "sub": "12345", "email": "...", "name": "...", "dept":
"...", ...}
```

5.3 Go(`golang-jwt/jwt v5`)

```
go get github.com/golang-jwt/jwt/v5
```

```go
package main

import (
    "crypto/rsa"
    "fmt"
    "net/http"
    "os"
    "time"

    "github.com/golang-jwt/jwt/v5"
)

var publicKey *rsa.PublicKey

func init() {
    keyBytes, err := os.ReadFile(os.Getenv("FOXLINK_GPT_PUBLIC_KEY_PATH"))
    if err != nil { panic(err) }
    publicKey, err = jwt.ParseRSAPublicKeyFromPEM(keyBytes)
    if err != nil { panic(err) }
}

func VerifyUserToken(r *http.Request) (jwt.MapClaims, error) {
    tokenStr := r.Header.Get("X-User-Token")
    if tokenStr == "" {
        return nil, fmt.Errorf("missing X-User-Token")
    }

    token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (interface{}, error) {
        if _, ok := t.Method.(*jwt.SigningMethodRSA); !ok {
            return nil, fmt.Errorf("unexpected signing method: %v",
t.Header["alg"])
        }
        return publicKey, nil
    },
        jwt.WithIssuer("foxlink-gpt"),
        jwt.WithLeeway(30*time.Second),
        jwt.WithValidMethods([]string{"RS256"}),  // 鎖死 RS256
    )

    if err != nil { return nil, err }
    if claims, ok := token.Claims.(jwt.MapClaims); ok {
```

```
        return claims, nil
    }
    return nil, fmt.Errorf("invalid claims")
}
```

5.4 C# / .NET(`System.IdentityModel.Tokens.Jwt`)

```
<PackageReference Include="System.IdentityModel.Tokens.Jwt" Version="7.*" />
```

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Cryptography;
using Microsoft.IdentityModel.Tokens;

public static class FoxlinkJwtVerifier
{
    private static readonly RsaSecurityKey _securityKey;

    static FoxlinkJwtVerifier()
    {
        var pem = File.ReadAllText(
            Environment.GetEnvironmentVariable("FOXLINK_GPT_PUBLIC_KEY_PATH")
            ?? "./foxlink-gpt-public.pem");
        var rsa = RSA.Create();
        rsa.ImportFromPem(pem);
        _securityKey = new RsaSecurityKey(rsa);
    }

    public static ClaimsPrincipal Verify(string token)
    {
        var validationParams = new TokenValidationParameters
        {
            ValidateIssuer          = true,
            ValidIssuer             = "foxlink-gpt",
            ValidateAudience        = false,
            IssuerSigningKey        = _securityKey,
            ValidAlgorithms         = new[] { "RS256" },  // 鎖死 RS256
            ClockSkew               = TimeSpan.FromSeconds(30),
        };
        var handler = new JwtSecurityTokenHandler();
        return handler.ValidateToken(token, validationParams, out _);
    }
}

// 使用
```

```
var principal = FoxlinkJwtVerifier.Verify(token);
var email = principal.FindFirst("email")?.Value;
```

5.5 Java(`jjwt`)

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
```

```
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.Claims;
import java.nio.file.*;
import java.security.*;
import java.security.spec.*;
import java.util.Base64;

public class FoxlinkJwtVerifier {
    private static final PublicKey PUBLIC_KEY = loadKey();

    private static PublicKey loadKey() {
        try {
            String pem = Files.readString(Paths.get(
                System.getenv().getOrDefault("FOXLINK_GPT_PUBLIC_KEY_PATH",
                    "./foxlink-gpt-public.pem")));
            String clean = pem.replace("-----BEGIN PUBLIC KEY-----", "")
                              .replace("-----END PUBLIC KEY-----", "")
                              .replaceAll("\\s", "");
            byte[] decoded = Base64.getDecoder().decode(clean);
            return KeyFactory.getInstance("RSA")
                .generatePublic(new X509EncodedKeySpec(decoded));
        } catch (Exception e) { throw new RuntimeException(e); }
```

```
    }

    public static Claims verify(String token) {
        return Jwts.parser()
            .verifyWith(PUBLIC_KEY)
            .requireIssuer("foxlink-gpt")
            .clockSkewSeconds(30)
            .build()
            .parseSignedClaims(token)
            .getPayload();
        // claims.get("email"), claims.get("sub"), claims.get("jti"), ...
    }
}
```

---

6. Transport-specific 注意事項

6.1 `http-post` 、 `streamable-http` 、 `http-sse`

標準 HTTP header, `X-User-Token` 直接從 request header 讀取。

6.2 `stdio`

stdio 子程序沒有 HTTP header → FOXLINK GPT 會透過**環境變數**傳遞:

```
MCP_API_KEY=<service api key>
MCP_USER_TOKEN=<RS256 JWT>
```

你的 stdio MCP 程式從 `process.env.MCP_USER_TOKEN` 讀取並驗簽。

> ⚠️ ***stdio 重要限制**:FOXLINK GPT 每次 `tools/call` 都會重新 spawn 一個新子程序(token 每次都是新鮮的)。**你的 stdio server 不可做 keep-alive / process pool** — 子程序常駐超過 5 分鐘,env var 裡的 token 就過期了(env var 一旦 spawn 就不能改)。*

---

7. 錯誤處理建議

情境	HTTP Status	行為
`X-User-Token` 缺失(且 admin 已開啟發送)	`401 Unauthorized`	`{"error":"missing user token"}`
JWT 簽章無效	`401`	可能公鑰版本不一致,紀錄 log
JWT `exp` 已過(超過 30s leeway)	`401`	FOXLINK GPT 每次呼叫都重簽,正常不該遇到
`iss ≠ "foxlink-gpt"`	`401`	防止其他服務偽用
`alg ≠ "RS256"`	`401`	**重要**:防 alg 切換攻擊

`email` 缺失	`403 Forbidden`	理論上不會發生,紀錄 log
使用者無該資料權限	`403`	正常業務邏輯,回 MCP tool error

MCP JSON-RPC level 可用 `error` object 回應:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32001,
    "message": "Unauthorized: invalid X-User-Token",
    "data": { "detail": "token expired at 2026-04-17T12:00:00Z" }
  }
}
```

---

8. 金鑰取得與 Rotation

8.1 取得公鑰

有三種方式:

1. **從 FOXLINK GPT Admin UI 下載**(推薦):登入 → MCP 伺服器管理 → 編輯任一 server → 使用者身份認證區塊 → 點「下載公鑰」→ 得到 `foxlink-gpt-public.pem`
2. **直接跟維運拿**: `rich_lai@foxlink.com` 索取
3. **參考本文件範例**(正式部署請以實際發給你的檔案為準):


```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAtwVX/aXN11Ozq+sW2nDV
jmqRdJHL7/0OQNrSY2gRTI23s/Lne/7HSnu1XXuVg1I5qALHW64zzZ9BrmeB8zj9
8hSozn75yN6Pr5Hxwo38ibYZTDp7NVYZYwdZSwyUca9itMTIdx6SSNvIoYewEsY1
juZwG2s+ee292kDpEAfPW/uah66K2O4VyRT8x7/6rv/ar+T1VNlRJXeWj8hCzCOG
L/RG9BtvX7q3qvd/ikFbl7h32jhachar/IjcMD4hPHPxwFtZT9HXvQTjI3C7MW1+
7hns/9u1SRDxote6UR5tDAjFuyYM+B60PYCu4XxIm2iOzRJKV7Qt9qkRywy0QHog
wwIDAQAB
-----END PUBLIC KEY-----
```


公鑰**可以公開**,洩漏不會造成安全問題(只能驗證,不能偽造簽名)。

8.2 Rotation 策略

預計 **1 年一次**或重大人事變動時會 rotate。屆時:

1. FOXLINK GPT 會**提前兩週**通知你新公鑰
2. 你的 MCP 需同時接受新舊兩把公鑰:

```
// Node.js 範例
const PUBLIC_KEYS = [
  fs.readFileSync('./foxlink-gpt-public.pem'),      // 舊
  fs.readFileSync('./foxlink-gpt-public-v2.pem'),   // 新
];
function verify(token) {
  for (const key of PUBLIC_KEYS) {
    try {
      return jwt.verify(token, key, { algorithms: ['RS256'], issuer:
'foxlink-gpt', clockTolerance: 30 });
    } catch (_) { continue; }
  }
  throw new Error('Invalid token');
}
```

3. FOXLINK GPT 切換私鑰後通知你,一週後你可移除舊公鑰

---

9. 測試與驗收

9.1 向 FOXLINK GPT 索取測試 token

方式一:跟 `rich_lai@foxlink.com` 要一顆測試 token

方式二:請 FOXLINK GPT admin 在 UI 上自助產:

> *MCP 伺服器管理 → 編輯任一 server → 「使用者身份認證」區塊 → 點「產生測試 Token」→ 填 email 後按「產生」→ 複製 JWT 給你*

9.2 FOXLINK GPT 提供的 CLI 驗證工具

如果你想**不依賴自己環境**先確認一顆 token 長怎樣,FOXLINK GPT 有一支 CLI:

```
node server/scripts/verify-mcp-token.js "<token>"
```

會印出 header、claims、 `iat` / `exp` 對應時間。可作為「我方驗證邏輯」的參考實作。

9.3 驗收 checklist

對接前請驗證以下項目:

- [ ] 收到公鑰 `foxlink-gpt-public.pem` ,存放於安全位置
- [ ] 收到測試 token 後,用公鑰驗簽成功
- [ ] 確認可讀到 `email` / `sub` / `name` / `dept` / `jti` 五個 claim
- [ ] **驗算過期拒絕**:人工塞一顆 `exp` 已過的 token → 被拒
- [ ] **驗證 iss 檢查**:人工簽一顆 `iss` 不是 `"foxlink-gpt"` 的 token → 被拒
- [ ] **驗證 alg 切換防護**:將 JWT header 改成 `{"alg":"HS256"}` 並用公鑰當 HMAC secret 簽 → **必須被拒**

- [ ] 設定 `clockTolerance` / `leeway` ≥ 30 秒
- [ ] 缺 `X-User-Token` header 時回 `401` (而非 `500`)
- [ ] (建議)log 紀錄 `jti` ,方便與 FOXLINK GPT `mcp_call_logs` 對齊
- [ ] stdio 模式(若有):從 `process.env.MCP_USER_TOKEN` 讀取,且**不常駐 process**

---

10. FAQ

Q1. Token 5 分鐘會不會太短? 對 HTTP 請求而言 5 分鐘遠夠用(request 從發送到回應通常 < 60s)。若某個 tool 執行超過 5 分鐘,也只影響 FOXLINK GPT 發出時那一刻的簽發,與你內部執行時間無關。

Q2. 我不想每個請求都驗簽(會影響效能),可以 cache 嗎? 可以。RS256 驗簽約 1ms,但若擔心可用 `jti` 做 short-TTL cache(例如 60s LRU)。但**不要 cache 超過 `exp` **。

Q3. FOXLINK GPT 會不會偽造其他使用者的身份打我? 理論上 FOXLINK GPT 自己有完整私鑰,有能力偽造。這需要信任 FOXLINK GPT 的 code review 和 access control 流程,本規格無法防禦。如果你的資料極度敏感,應在 MCP 端額外要求二次認證(例如 SSO)。

Q4. 我有多個 MCP server,可以共用同一把公鑰嗎? 可以。FOXLINK GPT 對所有 MCP server 都用同一把私鑰簽 token。每個 MCP 自己持有相同的公鑰即可。

Q5. 可以用 JWKS endpoint 自動取公鑰嗎? 目前(v1.0)不提供。未來若 MCP 數量成長到手動分發不實際,FOXLINK GPT 會提供 `/.well-known/jwks.json` 。屆時另行通知。

Q6. 我可以要求 FOXLINK GPT 在 token 中多塞欄位嗎(例如 factory_code / role)? 可以提需求,但目前建議**用 `email` 去查 AD** 取得這些資訊,比依賴 token 內的快照更即時準確。

Q7. api_key 如何管理? 你(MCP 開發者)自行產生一串 random string,交給 FOXLINK GPT admin 在 UI 上設定。建議 rotate 週期 6 個月。api_key 一把一個 MCP server 獨立,方便 revoke。

---

11. 對接流程建議

1. **Kickoff**:跟 FOXLINK GPT 端(`rich_lai@foxlink.com`)確認:
    - 你的 MCP 用哪種 transport
    - 你的 URL / 帶 api_key 格式
    - 索取公鑰 + 一顆測試 token
2. **實作 + 自測**:
    - 用 §9.3 checklist 逐項驗證
    - 單元測試涵蓋過期、alg 切換、iss 錯誤三種 negative case
3. **聯合測試**:
    - FOXLINK GPT admin 在 UI 新增你的 MCP server,**先不勾** send_user_token
    - 確認 service-level 通(`Authorization: Bearer`)
    - 再勾上 send_user_token,確認 user-level 通
4. **上線**:
    - FOXLINK GPT 發送線上測試 token 給你確認

- 正式在 UI 勾啟用

---

12. 聯絡資訊

項目	聯絡
技術對接 / 索取公鑰 / 測試 token	`rich_lai@foxlink.com`
公鑰 rotation 通知	同上,會在 rotate 前 2 週主動通知
發現 bug / 安全問題	同上,標註 `[MCP-SEC]` 於信件主旨

附錄 A:完整 Node.js 範例 server

這是一個最小可運作的 MCP server,可作為 onboarding 模板:

```
// minimal-mcp-server.js
const http = require('http');
const fs   = require('fs');
const jwt  = require('jsonwebtoken');

const PUBLIC_KEY = fs.readFileSync(
  process.env.FOXLINK_GPT_PUBLIC_KEY_PATH || './foxlink-gpt-public.pem'
);
const SERVICE_API_KEY = process.env.MCP_API_KEY || 'your-service-api-key';

function verifyService(req) {
  const h = req.headers['authorization'];
  if (!h?.startsWith('Bearer ')) return false;
  return h.slice(7) === SERVICE_API_KEY;
}

function verifyUser(token) {
  return jwt.verify(token, PUBLIC_KEY, {
    algorithms: ['RS256'],
    issuer: 'foxlink-gpt',
    clockTolerance: 30,
  });
}

const TOOLS = [{
  name: 'my_tool',
  description: 'Example tool',
  inputSchema: { type: 'object', properties: { query: { type: 'string' } } },
}];
```

```
http.createServer(async (req, res) => {
  // Layer 1: service auth
  if (!verifyService(req)) {
    res.writeHead(401, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify({ jsonrpc: '2.0', error: { code: -32001,
message: 'invalid api key' } }));
  }

  let body = '';
  for await (const c of req) body += c;
  const msg = JSON.parse(body);

  // Layer 2: user auth(只對 tools/call 強制)
  let userClaims = null;
  if (msg.method === 'tools/call') {
    const token = req.headers['x-user-token'];
    if (!token) {
      res.writeHead(401);
      return res.end(JSON.stringify({ jsonrpc: '2.0', id: msg.id, error: { code:
-32002, message: 'missing X-User-Token' } }));
    }
    try {
      userClaims = verifyUser(token);
    } catch (e) {
      res.writeHead(401);
      return res.end(JSON.stringify({ jsonrpc: '2.0', id: msg.id, error: { code:
-32003, message: `invalid token: ${e.message}` } }));
    }
  }

  let result;
  if (msg.method === 'initialize') {
    result = { protocolVersion: '2024-11-05', capabilities: { tools: {} },
serverInfo: { name: 'my-mcp', version: '1.0' } };
  } else if (msg.method === 'tools/list') {
    result = { tools: TOOLS };
  } else if (msg.method === 'tools/call') {
    // 判權範例:只允許 IT 部門用
    if (userClaims.dept !== 'IT-01') {
      result = { content: [{ type: 'text', text: 'Access denied (dept != IT-01)'
}], isError: true };
    } else {
      result = { content: [{ type: 'text', text: `Hello ${userClaims.name}, you
queried: ${msg.params.arguments?.query}` }] };
    }
    console.log(`[audit] jti=${userClaims.jti} email=${userClaims.email}
```

```
tool=${msg.params.name}`);
  }

  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ jsonrpc: '2.0', id: msg.id, result }));
}).listen(3000, () => console.log('Minimal MCP server on :3000'));
```

文件結束